



SDN using EVE

TSM - Advanced Communication Architecture

Dimitri Julmy

-

University of Applied Sciences and Arts of Western Switzerland (HES-SO)
Master of Science in Engineering (MSE)
Information and cyber security (ICS)

Repository URI	https://github.com/HezelTm/tsm_advcomarc
Creation date	2026-05-21
Submission date	2026-05-22

Introduction

TODO

Implementation

Mise en place EVE

La procédure ci-dessous est détaillée pour un environnement Arch Linux OS.

Installer VirtualBox (<https://wiki.archlinux.org/title/VirtualBox>):

```
1 yay -S virtualbox
```

Télécharger *Free EVE Community Edition Version 6.2.0-4*.

Créer une nouvelle VM (machine → New, Ctrl + n) avec les paramètres suivants, puis cliquer sur Finish :

- *Virtual machine name and operating system* :
 - VM Name : EVE-NG
 - ISO Image : /path/to/image/eve-ce-prod-6.2.0-4-full.iso
 - OS : Linux
- *Specify virtual hardware*
 - Base Memory : 8000 MB
 - CPU : 4
- *Specify virtual hard disk*
 - Disk Size : 50 GB

Dans Settings (mode Expert) → Network, passer l'Adapter 1 en Bridged Adapter. Laisser les autres paramètres par défaut.

Au premier démarrage, suivre l'installation de EVE-NG Community 6.2.0-4 (langue et clavier). Une fois terminée, la VM retourne sur GRUB : retirer le disque via Device → Optical Device → Remove Disk From Virtual Device, puis éteindre la machine (Power off the machine). Au redémarrage, se connecter avec :

- Nom d'utilisateur : root
- Mot de passe : eve

Conserver les valeurs par défaut pour les configurations suivantes. La machine redémarre ; se reconnecter avec les mêmes identifiants. L'URL de l'interface web est affichée dans la console (<http://10.24.45.108> dans notre cas). Se connecter avec les identifiants suivant :

- Nom d'utilisateur : admin
- Mot de passe : eve

Télécharger l'image du routeur *Arista vEOS-lab-4.36.0.1F.qcow2* et le *bootloader* *Aboot-veos-serial-8.0.2.iso* depuis le support de cours Moodle.

Avant de démarrer la VM, activer la virtualisation imbriquée (*nested virtualization*) dans VirtualBox pour que KVM soit disponible dans le guest (VM éteinte) :

```
1 # Sur la machine hôte (VM éteinte)
2 VBoxManage modifyvm "EVE-NG" --nested-hw-virt on
```

Démarrer la VM, puis transférer les images et corriger les permissions :

```
1 # Sur la machine hôte
2 ssh root@10.24.45.108 "mkdir -p /opt/unetlab/addons/qemu/veos-4.36.0.1F"
3 scp ~/Downloads/vEOS-lab-4.36.0.1F.qcow2 root@10.24.45.108:/opt/unetlab/addons/qemu/
veos-4.36.0.1F/hda.qcow2
4 scp ~/Downloads/Aboot-veos-serial-8.0.2.iso root@10.24.45.108:/opt/unetlab/addons/qemu/
veos-4.36.0.1F/cdrom.iso
5
6 # Sur la VM
7 echo "kvm" > /opt/unetlab/platform
8 /opt/unetlab/wrappers/unl_wrapper -a fixpermissions
```

Source : Youtube - TechNPlay - How to Install EVE-NG on Oracle VirtualBox 2025

EVE and alternate solutions

While installing EVE, please look for the differences between EVE and another alternate solution: GNS3, and document the findings.

GNS3 :

- Émulateur réseau *open-source*.
- Exécute de vraies images *IOS* (pas de logiciels simulés).
- Le comportement des équipements est identique au matériel physique (grâce aux vraies images *IOS*).
- Architecture client-serveur (l'interface graphique tourne localement tandis que le *backend* d'émulation tourne localement ou sur un serveur distant).

+ Avantages :

- Plus grande communauté (davantage de tutoriels, *templates* et ressources)
- Expérience bureau native
- Intégration officielle *Cisco*
- Excellente intégration *Wireshark*
- Développement actif

- Désavantages :

- Architecture mono-utilisateur (non conçue pour les simulations partagées)
- La configuration de la VM GNS3 peut être complexe
- Émulation *IOS legacy* uniquement pour les images anciennes

EVE :

- Plateforme d'émulation réseau fonctionnant entièrement comme une application serveur *Linux*.
- Accessible via un navigateur web (aucune installation locale requise).
- Conçue pour les environnements de labs partagés (plusieurs utilisateurs peuvent se connecter simultanément au même serveur *EVE-NG*, chacun travaillant dans sa propre topologie).
- Deux éditions : *Community* (gratuite) et *Professional* (payante)

+ Avantages :

- Accès via navigateur

- Environnements de labs partagés
- Fort support multi-constructeurs
- Déduplication mémoire *UKSM* (exécuter 10 routeurs identiques consomme beaucoup moins de RAM que 10 processus séparés)

- Désavantages :

- Configuration d'un serveur requise
- Les meilleures fonctionnalités sont payantes
- Communauté plus petite que *GNS3*

Comparaison des fonctionnalités

Fonctionnalité	GNS3	EVE-NG Community	EVE-NG Pro
Coût	Gratuit	Gratuit	\$120/an
Interface	Interface bureau (Windows/Mac/Linux)	Navigateur web	Navigateur web
Installation	Application locale + VM GNS3	Serveur <i>Linux</i> (bare metal ou VM)	Serveur <i>Linux</i>
Support multi-utilisateurs	✗ Mono-utilisateur	✗ Limité	☑ Multi-utilisateurs complet
Support <i>Cisco IOS</i>	☑ Complet	☑ Complet	☑ Complet
<i>Cisco IOS-XE/XR/NX-OS</i>	☑	☑	☑
<i>Palo Alto / Fortinet / Juniper</i>	☑ Via <i>QEMU</i>	☑ Via <i>QEMU</i>	☑ Via <i>QEMU</i>
Efficacité RAM	Standard	Standard	☑ Déduplication <i>UKSM</i>
Support conteneurs <i>Docker</i>	☑	☑	☑
Capture de paquets intégrée	☑ Intégration <i>Wireshark</i>	☑	☑
Communauté et templates	Très grande	Grande	Grande + officielle
Labs certification <i>Cisco</i>	☑ <i>GNS3 Academy</i> officielle	Labs communautaires	Labs <i>EVE-NG</i> officiels

Source : [ragheddris.com - EVE-NG vs GNS3](https://www.ragheddris.com/2026/03/13/eve-ng-vs-gns3-comparison/) : <https://www.ragheddris.com/2026/03/13/eve-ng-vs-gns3-comparison/>

Analyse de l'environnement EVE

Take a look at the tools and configurations made available by the program.

EVE-NG (Emulated Virtual Environment) expose une interface web permettant de concevoir, démarrer et superviser des topologies réseau entièrement depuis un navigateur.

Outils principaux :

- **Topology Editor** : éditeur graphique drag-and-drop pour créer des topologies réseau. Permet d'ajouter des nœuds, de les relier par des liens virtuels et d'exporter/importer des labs au format .unl.
- **Console d'accès** : accès aux nœuds via Telnet, SSH ou VNC selon le type d'équipement (IOL, QEMU, Docker).
- **Capture de paquets** : capture intégrée sur n'importe quel lien de la topologie, avec ouverture directe dans Wireshark sur la machine cliente.
- **Gestion des labs** : création, sauvegarde et partage de labs sous forme d'archives ZIP.

Configurations disponibles :

- **Templates de nœuds** : modèles préconfigurés pour une multitude de constructeurs (Cisco, Juniper, Arista, Palo Alto, Fortinet, etc.). Chaque template définit le type d'hyperviseur, le nombre d'interfaces, la RAM et le CPU alloués.
- **Types de réseaux** : réseau de management (Cloud0), bridges internes (Net) et connexions vers le réseau hôte (Cloud1–Cloud9).
- **Configurations de démarrage** : possibilité de pré-charger une configuration initiale sur chaque nœud avant le démarrage du lab.
- **Gestion des utilisateurs** : comptes utilisateurs avec rôles (administrateur, étudiant) et accès simultanés aux labs en édition Pro.

Source : [eve-ng.net](https://www.eve-ng.net/index.php/documentation/) - Documentation officielle EVE-NG : <https://www.eve-ng.net/index.php/documentation/>

Please check the infrastructure and librairies made available by EVE.

EVE-NG s'appuie sur un socle Linux (Ubuntu Server 22.04 LTS) et tire parti de plusieurs technologies de virtualisation pour émuler des équipements réseau hétérogènes.

Infrastructure :

- **KVM/QEMU** : hyperviseur principal utilisé pour exécuter les images QEMU des équipements réseau (routeurs, firewalls, switches). Chaque nœud tourne dans une VM légère.
- **IOL (IOS on Linux)** : permet de faire tourner certaines images Cisco IOS directement comme des processus Linux, sans virtualisation complète, ce qui réduit la consommation de ressources.
- **Docker** : support des conteneurs pour des nœuds légers (serveurs Linux, outils de test réseau).
- **Linux bridges / veth / tap** : interconnexion des nœuds via des interfaces réseau virtuelles du noyau Linux.
- **UKSM (Ultra Kernel Samepage Merging)** : mécanisme de déduplication mémoire qui fusionne les pages RAM identiques entre VMs, réduisant considérablement la RAM consommée lorsque plusieurs instances du même OS tournent simultanément (disponible en édition Pro).

API et bibliothèques :

- **EVE-NG REST API** : API HTTP complète pour piloter EVE-NG par programmation (gestion des labs, démarrage/arrêt des nœuds, configuration des topologies). Les principaux endpoints sont /api/labs, /api/nodes, /api/networks et /api/topologies.
- **peve** : bibliothèque Python communautaire encapsulant l'API REST d'EVE-NG, facilitant l'automatisation des labs depuis des scripts Python.

Source : [eve-ng.net](https://www.eve-ng.net/index.php/documentation/howtos/how-to-eve-ng-api/) - EVE-NG REST API : <https://www.eve-ng.net/index.php/documentation/howtos/how-to-eve-ng-api/>

Please check the website of ARISTA, a provider of SDN solutions. Try looking for libraries and extensions that will allow enriching the palette offered by natively by EVE.

Arista Networks est un fournisseur de solutions SDN dont le système d'exploitation **EOS (Extensible Operating System)** est conçu pour être entièrement programmable. Arista propose plusieurs bibliothèques et outils permettant d'enrichir les capacités d'EVE-NG.

Image virtuelle pour EVE-NG :

- **vEOS-lab** : image virtuelle d'EOS distribuée par Arista, compatible avec le moteur QEMU d'EVE-NG. Elle permet d'intégrer des switches/routeurs Arista réels dans les topologies EVE-NG pour tester les configurations EOS en lab.

Bibliothèques et extensions SDN :

- **eAPI** : API REST native d'EOS, exposée en HTTP/HTTPS sur chaque équipement Arista. Elle permet d'envoyer des commandes EOS et de récupérer l'état du réseau au format JSON, sans agent tiers.
- **pyeapi** : bibliothèque Python cliente pour eAPI, simplifiant l'automatisation des équipements Arista depuis des scripts Python (requêtes, configuration, parsing des résultats).
- **Arista AVD (Ansible Validated Designs)** : collection Ansible complète fournissant des rôles et playbooks pour concevoir, valider et déployer automatiquement des infrastructures réseau Arista (notamment des fabrics EVPN/VXLAN). Publiée sur Ansible Galaxy et GitHub.
- **EOS SDK** : SDK C++ permettant de développer des agents personnalisés qui s'exécutent nativement sur EOS, avec accès direct aux tables de routage, aux interfaces et aux événements réseau.
- **CloudVision (CVP)** : plateforme centralisée de gestion et de télémétrie réseau, offrant du streaming telemetry en temps réel et un contrôle de configuration réseau à grande échelle.

Source : avd.arista.com - Arista AVD : <https://avd.arista.com/>

Source : pyeapi.readthedocs.io - pyeapi documentation : <https://pyeapi.readthedocs.io/>

Source : [arista.com](https://arista.com/en/support/product-documentation/eos-sdk) - EOS SDK : <https://arista.com/en/support/product-documentation/eos-sdk>

Implement the architecture

La topologie réseau à implémenter est représentée par Figure 1.

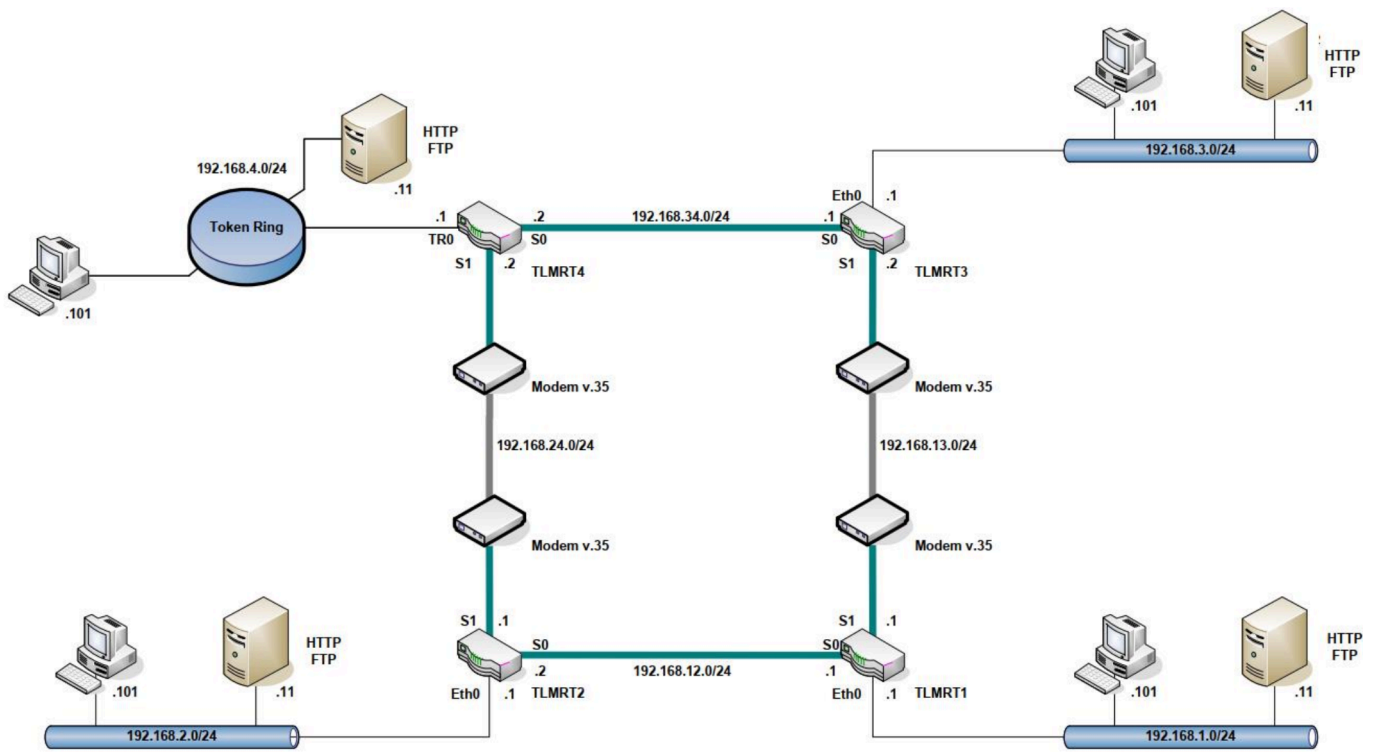


Figure 1: TODO

TODO

Conclusion

TODO